

INTRODUCCIÓN A LAS TELECOMUNICACIONES

Técnicas de transmisión de datos

Unidad 5 · Resumen detallado para el parcial

Resumen de **todas** las diapositivas del PDF *Telecomunicaciones_U5* (Universidad de Palermo), ordenado por temas y con diagramas para estudiar. Al final tenés cuadros comparativos, un glosario y una lista de puntos clave.

Código de cuadros: **OJO CON ESTO** = lo más probable de que lo pregunten · **PARA RECORDAR** = síntesis · **COMPLEMENTO** = explicación agregada por mí que no está en las diapositivas (sirve para entender, pero fijate qué te pide tu cátedra).

Mapa de la unidad

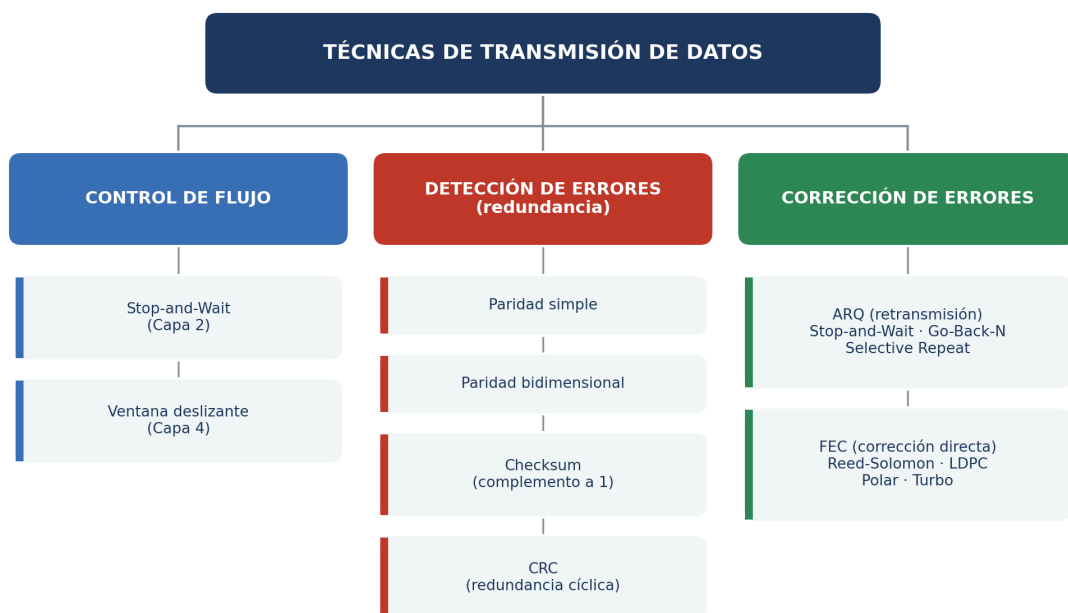


Figura 1. Esquema general de los temas de la unidad.

La unidad responde a tres preguntas: **(1)** cómo evitar que el emisor sature al receptor (*control de flujo*), **(2)** cómo darse cuenta de que un dato llegó dañado (*detección de errores*) y **(3)** cómo repararlo (*corrección de errores*: pidiendo que se reenvíe con ARQ, o corrigiéndolo directamente en el receptor con FEC).

1. Control de flujo de datos: 7 elementos clave

El **control de flujo** coordina la transmisión entre emisor y receptor para que el receptor no quede sobrecargado. El problema aparece cuando el emisor es más rápido que el receptor: aunque el enlace no tenga ningún error, si el receptor no puede procesar a la misma velocidad, se pierden datos.

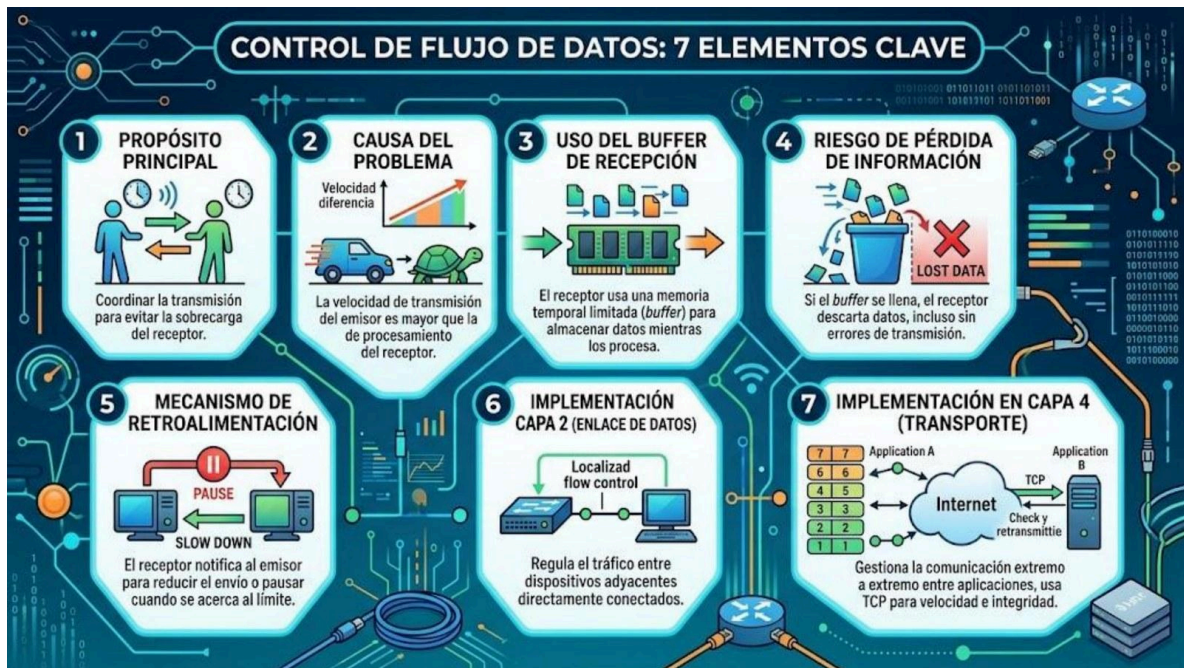


Figura 2. Infografía original de la cátedra: los 7 elementos clave del control de flujo.

Nº	Elemento	Qué hay que saber
1	Propósito principal	Coordinar la transmisión para evitar la sobrecarga del receptor .
2	Causa del problema	La velocidad de transmisión del emisor es mayor que la velocidad de procesamiento del receptor (diferencia de velocidad; la infografía lo ilustra con un auto y una tortuga).
3	Uso del buffer de recepción	El receptor usa una memoria temporal limitada (buffer) para almacenar datos mientras los procesa.
4	Riesgo de pérdida de información	Si el buffer se llena , el receptor descarta datos, incluso sin errores de transmisión (LOST DATA) .
5	Mecanismo de retroalimentación	El receptor notifica al emisor para que reduzca el envío o pause cuando se acerca al límite (mensajes PAUSE / SLOW DOWN).
6	Implementación en Capa 2 (Enlace de datos)	Control de flujo localizado: regula el tráfico entre dispositivos adyacentes directamente conectados .
7	Implementación en Capa 4 (Transporte)	Gestiona la comunicación extremo a extremo entre aplicaciones . Usa TCP para velocidad e integridad (verifica y retransmite).

OJO CON ESTO · Capa 2 vs. Capa 4

Capa 2 (enlace): control **local** entre equipos vecinos (por ejemplo, PC ↔ switch). En la unidad se ve con **Stop-and-Wait**.

Capa 4 (transporte): control **extremo a extremo** entre aplicaciones (Application A ↔ Internet ↔ Application B). En la unidad se ve con la **ventana deslizante de TCP**.

Un buffer lleno provoca pérdida de datos **aunque no haya errores de transmisión**: por eso el control de flujo es un problema distinto del control de errores.

2. Stop and Wait (Capa 2)

Es el mecanismo de control de flujo más simple: el emisor envía **un mensaje por vez** y no manda el siguiente hasta recibir la confirmación. Funciona en 3 pasos:

- 1.- **Envío y retención:** el emisor envía el mensaje y **lo conserva en su buffer** como respaldo.
- 2.- **Recepción y confirmación:** el receptor guarda o procesa el mensaje y responde enviando un **ACK** (confirmación).
- 3.- **Liberación y avance:** el emisor recibe el ACK, **elimina el mensaje guardado en su buffer** y transmite el siguiente.

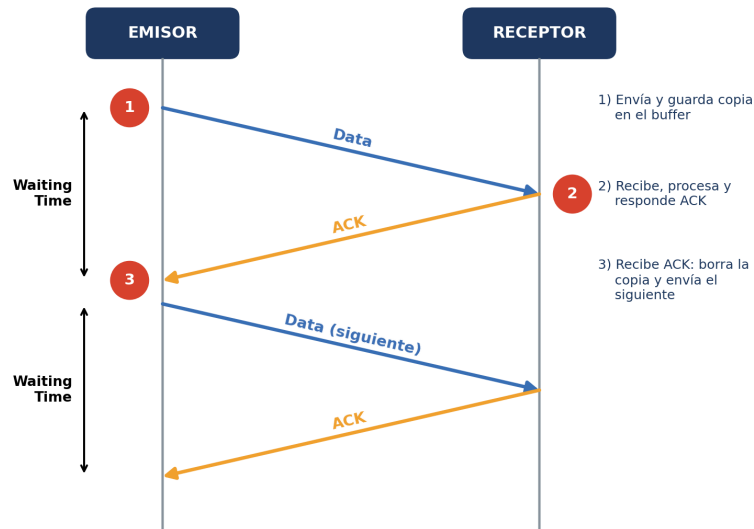


Figura 3. Protocolo Stop-and-Wait: cada Data espera su ACK (Waiting Time).

COMPLEMENTO (no está en las diapositivas) · Limitación

En el diagrama original se marca el **Waiting Time**: mientras espera el ACK, el emisor **no transmite nada**. Con enlaces largos o lentos se desaprovecha mucho tiempo. Ese es el problema que resuelve la **ventana deslizante** (sección 4), que permite tener varios mensajes "en vuelo".

3. Stop and Wait ARQ (Capa 2): qué pasa cuando algo sale mal

Stop-and-Wait también sirve para **controlar errores**. Para eso suma tres herramientas: un **temporizador (timeout)** que el emisor arranca cada vez que envía, los mensajes de respuesta **ACK** (llegó bien) y **NAK** (llegó con error), y un **bit de control (C = 0/1)** que numera los mensajes para distinguir un mensaje nuevo de una retransmisión.

Las diapositivas presentan tres situaciones posibles:

3.1 La trama no llega al receptor

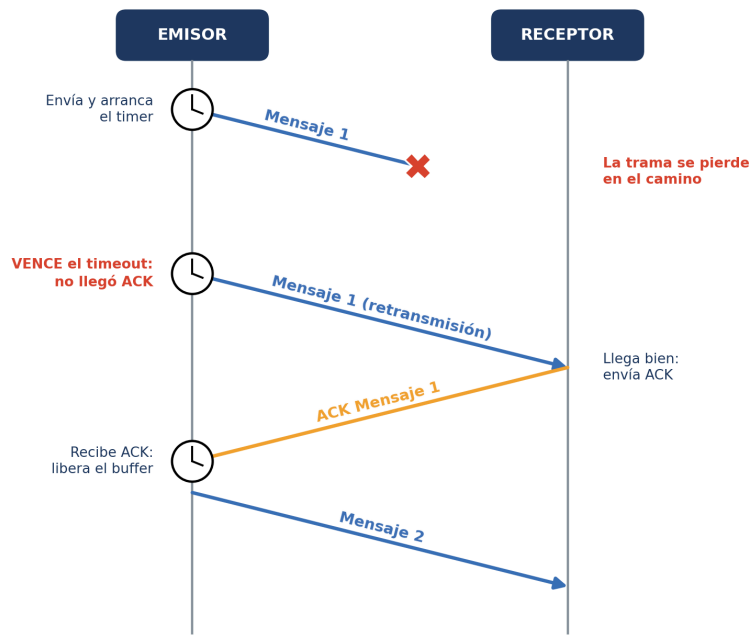


Figura 4. Stop-and-Wait ARQ: la trama se pierde.

- El emisor envía el **Mensaje 1** y arranca el timer; el mensaje **se pierde** en el camino.
- Como no llega ningún ACK, **vence el timeout** y el emisor **retransmite el Mensaje 1**.
- El receptor lo recibe bien y responde **ACK Mensaje 1**; el emisor libera el buffer y envía el **Mensaje 2**.

3.2 El ACK no llega al emisor

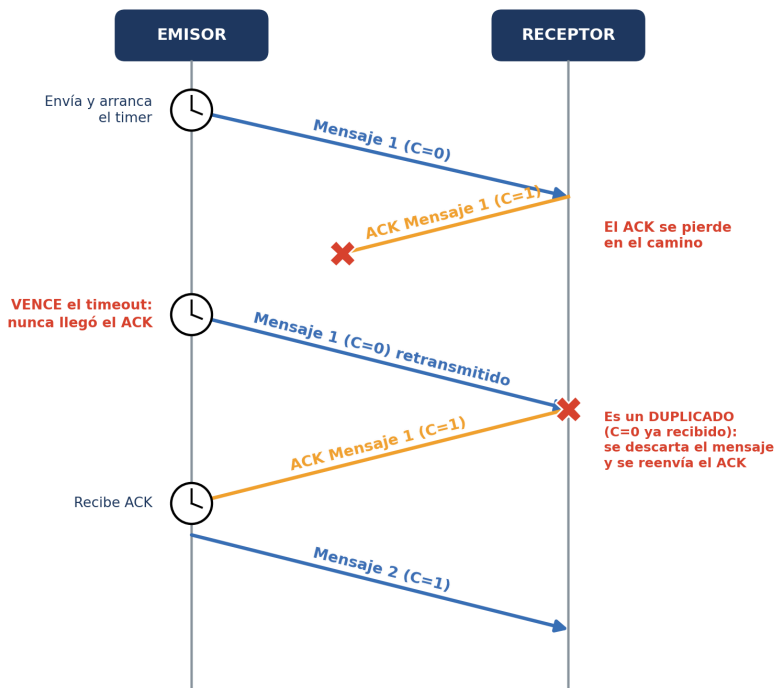


Figura 5. Stop-and-Wait ARQ: el ACK se pierde.

- El receptor recibe bien el **Mensaje 1 (control = 0)** y responde **ACK (C = 1)**, pero el **ACK se pierde**.
- El emisor no sabe que el mensaje llegó: **vence el timeout** y **retransmite el Mensaje 1 (C = 0)**.

- El receptor ya tenía ese mensaje: por el bit de control detecta que es un **duplicado**, lo **descarta** (“Se descarta el mensaje”) y **vuelve a enviar el ACK**.
- El emisor recibe el ACK y sigue con el **Mensaje 2 (C = 1)**.

OJO CON ESTO

Sin el bit de control (número de secuencia), el receptor **entregaría el mensaje dos veces** a la capa superior. Por eso la retransmisión necesita poder identificarse como duplicado.

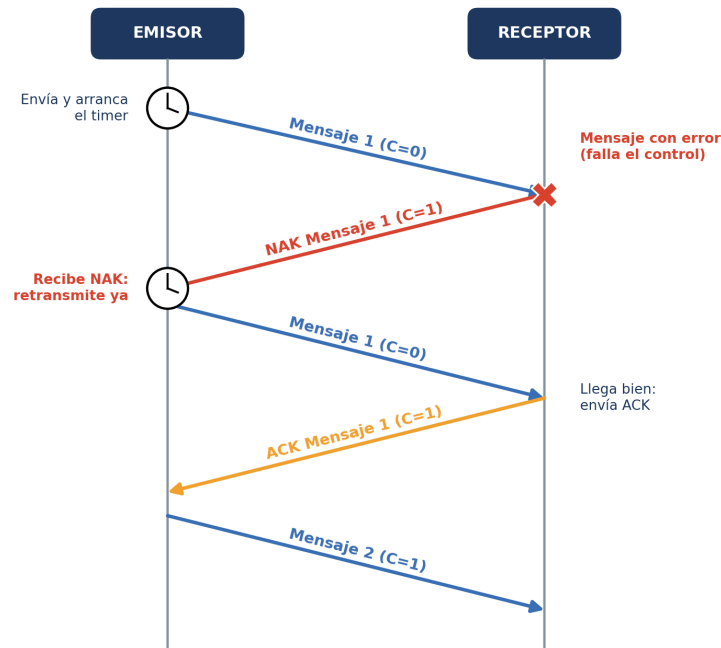
3.3 El mensaje llega con errores

Figura 6. Stop-and-Wait ARQ: el mensaje llega con errores.

- El **Mensaje 1 (control = 0)** llega **con error** (se detecta con los códigos de detección de errores de la sección 5).
- El receptor responde un **NAK Mensaje 1** (aviso negativo) en lugar de un ACK.
- Al recibir el NAK, el emisor **retransmite el Mensaje 1** (sin esperar al timeout, según el diagrama), llega bien, recibe el **ACK** y envía el **Mensaje 2**.

Resumen de los tres casos

Problema	Quién lo detecta	Cómo se detecta	Qué se hace
Trama perdida	Emisor	Vence el timeout sin recibir ACK	Retransmite la trama
ACK perdido	Emisor (por timeout) y receptor (duplicado)	Timeout en el emisor; bit de control repetido en el receptor	Emisor retransmite; receptor descarta el duplicado y reenvía el ACK
Mensaje con errores	Receptor	Falla la verificación de errores	Envía NAK ; el emisor retransmite

4. Ventana deslizante (Capa 4)

Con la ventana deslizante el emisor **no espera un ACK por cada segmento**: puede enviar varios segmentos seguidos, hasta completar el tamaño de la **ventana**. A medida que llegan ACKs, la ventana se **desliza** hacia adelante y habilita el envío de datos nuevos. En TCP la ventana se mide en **bytes** y el receptor la usa para regular cuánto puede mandarle el emisor (control de flujo).

4.1 Simulador “TCP Sliding Window”

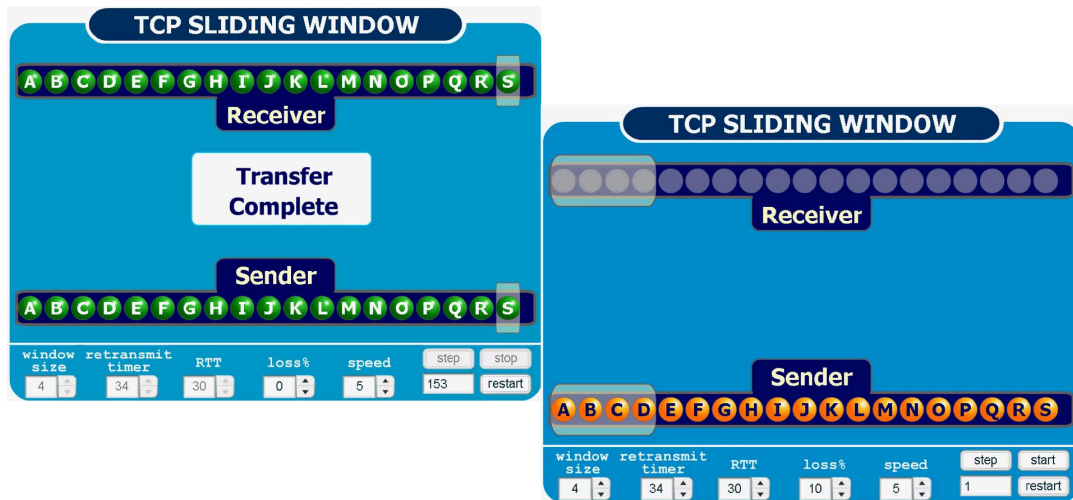


Figura 7. Simulador de ventana deslizante TCP de la cátedra (izquierda: transferencia completa; derecha: estado inicial).

- Parámetros configurables: **window size = 4**, **retransmit timer = 34**, **RTT = 30**, **loss % (porcentaje de pérdida)** y **speed = 5**.
- El **Sender** (abajo) tiene los paquetes A–S; el **Receiver** (arriba) los va recibiendo. El recuadro semitransparente es la **ventana** (4 paquetes) y se va corriendo.
- **Izquierda**: con **loss = 0 %** aparece “**Transfer Complete**” (contador en 153): todos los paquetes llegaron.
- **Derecha**: estado inicial de otra corrida con **loss = 10 %**: al haber pérdidas, el emisor tiene que retransmitir cuando vence el **retransmit timer**.

4.2 Captura de Wireshark: la ventana en un segmento real

```

▶ Frame 17189: 227 bytes on wire (1816 bits), 227 bytes captured (1816 bits) on interface \Device\NPF_{80D32880-A61C-4C07-BE06-3CABFD8BDEE6}, id 0
▶ Ethernet II, Src: ASUSTekCOMPU_bb:6c:9b (08:bf:b8:bb:6c:9b), Dst: TpLinkTechno_ef:25:a6 (e8:94:f6:ef:25:a6)
▶ Internet Protocol Version 4, Src: 172.16.2.230, Dst: 172.16.100.24
▼ Transmission Control Protocol, Src Port: 58869, Dst Port: 1900, Seq: 1, Ack: 1, Len: 173
  Source Port: 58869
  Destination Port: 1900
  [Stream index: 22]
  [Stream Packet Number: 4]
  ▶ [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 173]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 9534600
  [Next Sequence Number: 174 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 97721930
  0101 .... = Header Length: 20 bytes (5)
  ▶ Flags: 0x018 (PSH, ACK)
  Window: 255
  [Calculated window size: 65280]
  [Window size scaling factor: 256]
  Checksum: 0xbfe6 [unverified]
  [Checksum Status: Unverified]

```

Figura 8. Captura de un segmento TCP en Wireshark; se resalta el tamaño de ventana calculado.

Campo	Valor en la captura	Qué significa
Frame 17189	227 bytes (1816 bits)	Tamaño de la trama capturada ($227 \times 8 = 1816$ bits).
Ethernet II (Capa 2)	Src ASUSTek 08:bf:b8:bb:6c:9b → Dst TpLink e8:94:f6:ef:25:a6	Direcciones MAC de origen y destino.
IPv4 (Capa 3)	Src 172.16.2.230 → Dst 172.16.100.24	Direcciones IP de origen y destino.
Source / Destination Port	58869 → 1900	Puertos TCP de origen y destino (Capa 4).
Sequence Number	1 (relativo); raw 9534600	Número del primer byte de datos del segmento. Next Sequence Number = 174 ($1 + 173$).
Acknowledgment Number	1 (relativo); raw 97721930	Próximo byte que se espera recibir del otro extremo.
TCP Segment Len	173	Bytes de datos del segmento.
Header Length	20 bytes (5) — “0101”	Largo del encabezado TCP: 5 palabras de 4 bytes.
Flags	0x018 (PSH, ACK)	ACK : el número de confirmación es válido. PSH : entregar los datos enseguida a la aplicación.
Window	255	Valor “crudo” del campo de ventana anunciado.
Window size scaling factor	256	Factor de escala negociado (permite ventanas mayores a 65 535 bytes).
Calculated window size	65280	$255 \times 256 = 65\,280$ bytes : cuántos bytes puede recibir sin necesidad de confirmar.
Checksum	0xbfe6 [unverified]	Suma de verificación de TCP (detección de errores, sección 5.4). Wireshark no la verificó.
Datos de Wireshark	Stream index 22 · Stream Packet Number 4 · Conversation completeness: Complete, WITH_DATA (31)	Campos entre corchetes: los calcula Wireshark , no viajan en el paquete.

COMPLEMENTO (no está en las diapositivas) · Chequeo del tamaño

227 bytes = 14 (Ethernet) + 20 (IP) + 20 (TCP) + 173 (datos). Y la ventana efectiva es el valor del campo (255) por el factor de escala (256).

4.3 Ejemplo paso a paso (segmentos de 1000 bytes, ventana de 3000 bytes)

Tras el *handshake* (establecimiento de la conexión), la ventana permite tener hasta **3000 bytes sin confirmar**. Los segmentos son A (SEQ = 1000), B (SEQ = 2000), C (SEQ = 3000), y así sucesivamente.

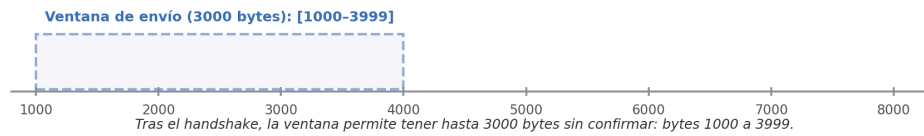
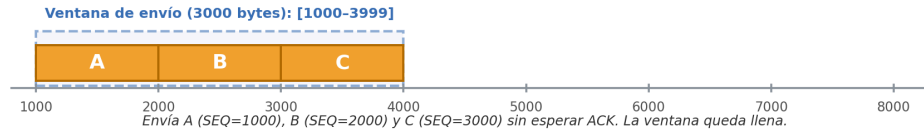
Paso 0 — Estado inicial**Paso 1 — El emisor llena la ventana****Paso 2 — Llega el primer ACK**

Figura 9. Pasos 0 a 2: estado inicial, la ventana se llena y llega el primer ACK.

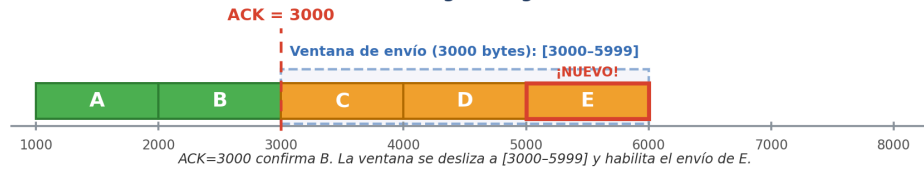
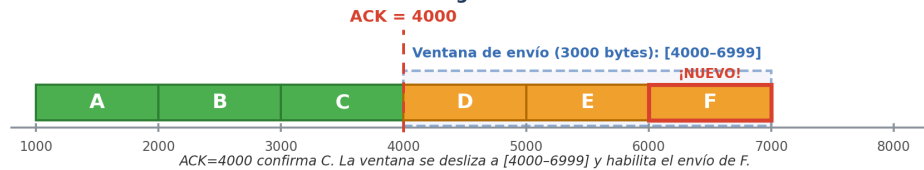
Paso 3 — La ventana se desliza**Paso 4 — Llega el segundo ACK****Paso 5 — Llega el tercer ACK**

Figura 10. Pasos 3 a 5: la ventana se desliza a medida que llegan los ACK.

Paso	Evento	Ventana de envío	Qué pasa
0	Estado inicial	[1000–3999]	Tras el handshake, se pueden tener hasta 3000 bytes sin confirmar.
1	El emisor llena la ventana	[1000–3999]	Envía A (SEQ=1000), B (SEQ=2000) y C (SEQ=3000).
2	Llega el primer ACK = 2000	[1000–3999]	Confirma el byte 1999 y todos los anteriores (A). El receptor espera ahora el byte 2000.

Paso	Evento	Ventana de envío	Qué pasa
3	La ventana se desliza	[2000–4999]	El borde izquierdo avanza a 2000 y el derecho a 4999. Se libera espacio: se envía D (SEQ=4000) .
4	Llega ACK = 3000	[3000–5999]	Confirma B. La ventana se desliza y habilita el envío de E (SEQ=5000) .
5	Llega ACK = 4000	[4000–6999]	Confirma C. La ventana se desliza y habilita el envío de F (SEQ=6000) .

OJO CON ESTO · Cómo leer el ACK

El **ACK indica el próximo byte que el receptor espera**: ACK = 2000 significa “recibí hasta el 1999, mandame desde el 2000”.

El **borde izquierdo** de la ventana queda en el último ACK recibido; el **borde derecho** es ese valor + tamaño de ventana – 1 (por eso, con ACK = 2000 y ventana de 3000, la ventana va de 2000 a 4999).

4.4 ACK acumulativos y SACK

- **ACK acumulativos**: TCP los implementa para **evitar tráfico innecesario en la red**. Un solo ACK confirma todo lo anterior (ej.: ACK = 2000 confirma el byte 1999 y todos los previos), sin necesidad de un ACK por cada segmento.
- **SACK (Selective ACK)**: el receptor **informa que recibió determinados bloques de información**, es decir, avisa qué bloques llegaron aunque haya huecos, para que el emisor reenvíe solo lo que falta.

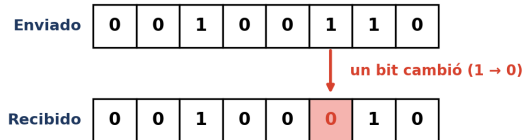
4.5 Ventajas más importantes

- **Mayor eficiencia en redes full-dúplex** (se puede enviar y recibir al mismo tiempo, y el enlace no queda ocioso esperando ACKs).
- **Piggyback para reducir el overhead**: el ACK “viaja a costas” de un segmento de datos que ya iba en sentido contrario, en lugar de mandar un paquete aparte solo para confirmar.

5. Detección y corrección de errores

5.1 Tipos de error y redundancia

Error de un solo bit (single bit error)



Error en ráfaga (burst error)



Figura 11. Error de un solo bit y error en ráfaga.

- **Single bit error (error de un solo bit):** cambia un bit. Ejemplo de la diapositiva: se enviaba 00100110 y se recibe 00100010.
- **Burst error (error en ráfaga):** se alteran varios bits. La **longitud de la ráfaga** se mide desde el **primer** hasta el **último** bit dañado (en el ejemplo, 8 bits, aunque solo 4 bits estén realmente corrompidos).

OJO CON ESTO · Idea central

Detección de errores → **redundancia:** se **agregan bits adicionales** al mensaje. El receptor los recalcula y, si no coinciden con lo recibido, sabe que hubo un error.

5.2 Control de paridad: paridad simple (par o impar)

Se agrega **1 bit de paridad** para que la cantidad total de unos sea **par** (paridad par) o **impar** (paridad impar).

Ejemplo de la diapositiva con **paridad par**:

Paso 1: el emisor quiere mandar 1011001 y cuenta los unos: hay **4**.

Paso 2: como usa paridad par y ya hay 4 (par), el bit de paridad es **0** (si hubiera 3 unos, se agregaría un 1 para mantener la paridad par). El mensaje transmitido es **10110010** (*codeword*).

Paso 3: el receptor cuenta los unos: hay 4. La regla es paridad par y se cumple, así que **asume que no hubo error**.

Con error: si se recibe 10111010, el receptor cuenta **5 unos**. Como la regla era paridad par, **detecta que hubo un error**.

Emisor: quiere mandar 1011001 (paridad PAR)

Dato	1	0	1	1	0	0	1	bit de paridad
								0

4 unos (par) → bit de paridad = 0
Codeword transmitida: 10110010

Receptor: recibe con un bit alterado

Recibido	1	0	1	1	1	0	1	0
----------	---	---	---	---	---	---	---	---

Recibe 10111010 → cuenta 5 unos (impar)
La regla era paridad par → ¡ERROR detectado!

Si recibe 10110010: cuenta 4 unos (par) → la regla se cumple → asume que no hubo error.

Figura 12. Paridad simple (par): emisor y receptor.

COMPLEMENTO (no está en las diapositivas) · Límites de la paridad simple

- Solo **detecta**: sabe que hubo un error, pero **no sabe qué bit** falló, así que no puede corregirlo.
- Detecta cuando se alteran una cantidad **impar** de bits. Si se alteran 2 bits, la cantidad de unos sigue siendo par y **el error pasa inadvertido**.

5.3 Control de paridad: paridad bidimensional

Los bloques se ordenan en filas y se calcula la paridad **por fila** (paridad horizontal) y **por columna** (paridad vertical). En la diapositiva se muestran los valores **recibidos**, con paridad par:

Bloque / Fila	Bit 1	Bit 2	Bit 3	Bit 4	Paridad Horiz.	Chequeo por fila
Bloque 1	1	0	1	1	1	✓ 4 unos
Bloque 2	0	1	0 (error)	0	0	✗ 1 uno → impar
Bloque 3	1	1	0	1	1	✓ 4 unos
Paridad Vert.	0	0	0	0	0	

Columnas: ✓ ✓ ✗ ✓ ✓

La fila 2 y la columna 3 fallan → su intersección (Bloque 2, Bit 3) es el bit erróneo.
Como se sabe cuál es, se puede corregir invirtiéndolo (0 → 1).

Figura 13. Paridad bidimensional: la intersección de la fila y la columna que fallan localiza el bit erróneo.

- Bloque 2** recibido: 0 1 **0 (error)** 0 y paridad horizontal 0 → hay 1 solo uno (impar) → **la fila 2 falla**.
- En la **columna del Bit 3** hay un solo 1 (más paridad vertical 0) → cantidad impar → **la columna 3 falla**.
- La **intersección** (Bloque 2, Bit 3) es el bit erróneo. Como se sabe cuál es, se puede **corregir** invirtiéndolo (0 → 1).

PARA RECORDAR

Paridad simple: detecta. **Paridad bidimensional:** detecta y, para errores de un solo bit, **localiza y corrige** (las filas y columnas que fallan se cruzan en el bit dañado). La contrapartida es más bits de redundancia.

COMPLEMENTO (no está en las diapositivas) · Nota sobre la lectura de la tabla

La diapositiva marca el error en el Bloque 2 / Bit 3; la interpretación de “qué fila y columna fallan” y la corrección la deduje aplicando la regla de paridad par a los valores mostrados.

5.4 Checksum (complemento a 1) y CRC (control de redundancia cíclica)

Las diapositivas los nombran como los otros dos métodos de detección de errores. Ambos se apoyan en el mismo principio: **agregar redundancia calculada a partir de los datos**.

COMPLEMENTO (no está en las diapositivas) · Cómo funcionan (resumen)

- **Checksum (complemento a 1):** el emisor suma los bloques del mensaje en aritmética de complemento a 1 y envía el **complemento de esa suma** como checksum. El receptor suma todo (incluido el checksum): si el resultado da todo unos (o su complemento da cero), asume que no hubo error. Es el que usan **IP/TCP/UDP** (en la captura, “Checksum: 0xbfe6”).
- **CRC:** el mensaje se trata como un polinomio y se **divide** por un polinomio generador que conocen ambos extremos; el **resto** se agrega al final de la trama. El receptor repite la división: si el resto es 0, no hay error. Detecta muy bien **errores en ráfaga** y se usa en Capa 2 (por ejemplo Ethernet).

6. Técnicas de corrección de errores

Una vez detectado el error hay dos estrategias: **pedir que se retransmita** (ARQ) o **corregirlo en el receptor** con redundancia extra (FEC).

6.1 ARQ (protocolos con retransmisión)

COMPLEMENTO (no está en las diapositivas)

ARQ = *Automatic Repeat reQuest* (solicitud automática de repetición).

Stop-and-Wait ARQ (Parada y Espera)

Según la diapositiva, es el **protocolo base más elemental** de control de errores.

- El emisor transmite **una sola trama a la vez** y **se detiene a esperar el ACK** del receptor antes de enviar la siguiente.
- Si **vence el temporizador (timeout)** sin recibir el ACK, **retransmite la trama actual**.

Go-Back-N ARQ (ARQ con Retroceso N)

- El emisor puede transmitir **múltiples tramas continuas (hasta N)** sin esperar un ACK inmediato.
- El receptor **solo acepta tramas en estricto orden secuencial**.
- Si una trama **se pierde o llega con error**, el receptor **la descarta junto con todas las tramas posteriores** (y el emisor “retrocede” N).

Video de la cátedra: <https://www.youtube.com/watch?v=gaocB7unrqs>

Selective Repeat ARQ (ARQ con Repetición Selectiva)

- El emisor transmite **múltiples tramas (hasta N)**.
- A diferencia de Go-Back-N, el receptor **acepta tramas fuera de orden** y las **almacena en un búfer temporal** hasta recibir la trama faltante, para **reordenarlas y entregarlas a la capa superior**.
- El emisor **solo retransmite la trama específica que falló** o cuyo temporizador expiró, **sin tocar las tramas ya confirmadas** correctamente.

Video de la cátedra: https://www.youtube.com/watch?v=6fz_Wo8jkAc

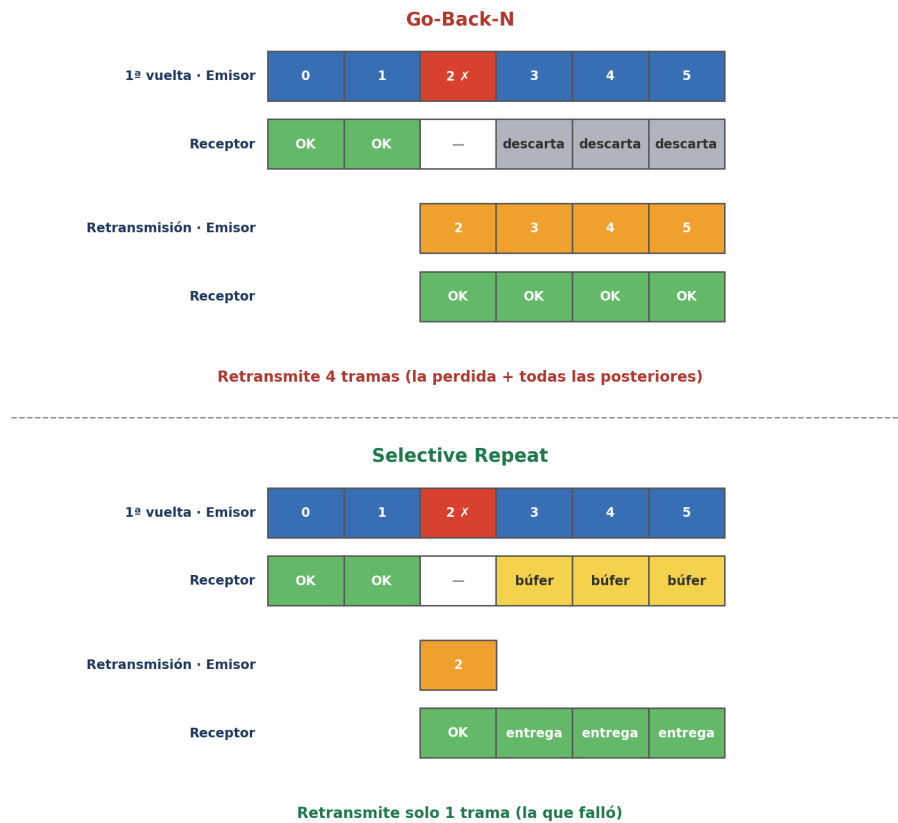


Figura 14. Misma pérdida (trama 2), tratada con Go-Back-N y con Selective Repeat.

Protocolo	Tramas en vuelo	Receptor acepta fuera de orden	Qué retransmite	Lectura (conclusión)
Stop-and-Wait	1	No aplica (una a la vez)	La trama actual (al vencer el timeout)	Simple, pero el emisor queda ocioso.
Go-Back-N	Hasta N	No: solo orden estricto	La trama perdida y todas las posteriores	Más rápido que S&W, pero reenvía tramas que sí habían llegado.
Selective Repeat	Hasta N	Sí: las guarda en un búfer	Solo la que falló o expiró	Aprovecha mejor el enlace; receptor más complejo (búfer y reordenamiento).

La última columna es una conclusión que se deduce del funcionamiento, no un texto literal de las diapositivas.

6.2 FEC (corrección directa de errores)

COMPLEMENTO (no está en las diapositivas)

FEC = Forward Error Correction.

- **Corrige los errores en el receptor utilizando bits de redundancia** (no hace falta pedir retransmisión).
- **Ventaja:** libera tráfico de red (evita retransmisiones).
- **Desventaja:** requiere **más procesamiento y ancho de banda** para la transmisión de los mensajes (los bits de redundancia viajan siempre).

OJO CON ESTO · Dónde se usa

- **FEC NO se utiliza en redes LAN:** se prefiere la **retransmisión**.
- **FEC SÍ se utiliza en redes inalámbricas y de gran distancia.**

Código FEC	Dónde / para qué se usa	Palabra clave
Reed-Solomon	FC (Fibre Channel) y DOCSIS. Redes de ráfagas.	Ráfagas
LDPC	Estándar para redes de alta velocidad. Ej.: TV satelital.	Alta velocidad
Polar Codes	5G.	5G
Turbo Codes	4G y espacio. Para señales débiles.	4G / espacio / señales débiles

COMPLEMENTO (no está en las diapositivas) · Por qué esta división

En una **LAN** el medio es confiable y de baja latencia, así que retransmitir sale barato y no vale la pena cargar todos los mensajes con redundancia. En enlaces **inalámbricos o de gran distancia** hay más errores y la retransmisión tarda mucho (ida y vuelta larga), por eso conviene corregir directamente en el receptor.

7. Cuadros comparativos y repaso

7.1 Stop-and-Wait vs. Ventana deslizante

Aspecto	Stop-and-Wait	Ventana deslizante
Capa (en las diapositivas)	Capa 2 (enlace de datos)	Capa 4 (transporte, TCP)
Mensajes sin confirmar	Uno solo	Varios, hasta llenar la ventana (ej.: 3000 bytes)
Comportamiento del emisor	Envía, guarda copia y espera el ACK (Waiting Time)	Sigue enviando mientras haya espacio; la ventana se desliza con cada ACK
Confirmaciones	Un ACK por mensaje	ACK acumulativos (y SACK)
Redes full-dúplex	Poco eficiente	Mayor eficiencia; piggyback reduce el overhead

Ojo: Go-Back-N y Selective Repeat aplican la misma idea de “ventana” pero contando **tramas** (hasta N) en lugar de bytes.

7.2 ARQ vs. FEC

Aspecto	ARQ (retransmisión)	FEC
Cómo corrige	El emisor retransmite lo que falló (por timeout o NAK)	El receptor corrige usando bits de redundancia
Ventaja	No agrega tanta redundancia a cada mensaje	Libera tráfico de red (no hay retransmisiones)
Desventaja	Genera tráfico y demoras por retransmitir	Más procesamiento y ancho de banda para transmitir los mensajes
Dónde se usa	Redes LAN (se prefiere retransmisión)	Redes inalámbricas y de gran distancia

Aspecto	ARQ (retransmisión)	FEC
Ejemplos	Stop-and-Wait, Go-Back-N, Selective Repeat	Reed-Solomon, LDPC, Polar, Turbo

La fila “Ventaja” del ARQ es una conclusión agregada; el resto sale de las diapositivas.

7.3 Métodos de detección de errores

Método	Qué hace	Detecta / corrige
Paridad simple (par o impar)	1 bit extra para que la cantidad de unos sea par o impar	Solo detecta (errores en cantidad impar de bits)
Paridad bidimensional	Paridad por fila y por columna	Detecta y localiza el bit erróneo (permite corregir 1 bit)
Checksum (complemento a 1)	Suma de bloques en complemento a 1	Detecta
CRC	Resto de una división polinómica agregado a la trama	Detecta (muy bueno con ráfagas)

7.4 Glosario rápido

Término	Definición
ACK	Confirmación positiva: “recibí bien”. En TCP indica el próximo byte esperado.
NAK	Confirmación negativa: “llegó con error, reenviá”.
Timeout / temporizador	Tiempo máximo que el emisor espera un ACK antes de retransmitir.
Buffer / búfer	Memoria temporal. En el emisor guarda la copia hasta el ACK; en el receptor guarda los datos mientras los procesa (y, en Selective Repeat, las tramas fuera de orden).
Trama / Segmento	Unidad de datos de Capa 2 (trama) y de Capa 4 (segmento TCP).
SEQ (número de secuencia)	Número del primer byte de datos de un segmento (ej.: SEQ = 2000).
Handshake	Intercambio inicial de TCP que establece la conexión y acuerda la ventana.
Ventana (window)	Cantidad de datos que se pueden tener enviados sin confirmar.
ACK acumulativo	Un ACK que confirma todos los bytes anteriores al número indicado.
SACK	ACK selectivo: informa qué bloques recibió el receptor.
Piggyback	Enviar el ACK dentro de un paquete de datos que va en sentido contrario.
Overhead	Información extra (encabezados, confirmaciones, redundancia) que no es dato útil.
Full-dúplex	Comunicación simultánea en ambos sentidos.
Redundancia	Bits adicionales agregados para detectar o corregir errores.
Codeword	Mensaje ya con los bits de redundancia agregados (ej.: 10110010).
Single bit / burst error	Error de un bit / error que abarca una ráfaga de varios bits.
ARQ / FEC	Corrección por retransmisión / corrección directa en el receptor.

7.5 Checklist: lo que no te puede faltar en el parcial

- **Control de flujo:** sirve para que el emisor no sature el **buffer** del receptor; hay pérdida de datos **aunque no haya errores de transmisión**. El receptor avisa (pausa / bajar velocidad).
- **Capa 2:** control local entre dispositivos adyacentes. **Capa 4:** control extremo a extremo entre aplicaciones (TCP).
- **Stop-and-Wait:** 3 pasos (envío y retención → recepción y ACK → liberación y avance). Si vence el timeout, retransmite.
- **Los 3 casos de Stop-and-Wait ARQ:** trama perdida (timeout), ACK perdido (timeout + el receptor descarta el duplicado y reenvía el ACK), mensaje con errores (NAK).
- **Ventana deslizante:** el ACK es el próximo byte esperado; la ventana se desliza (ejemplo: 3000 bytes, ACK 2000 → ventana [2000–4999] → se envía D).
- **ACK acumulativos** (evitan tráfico innecesario) y **SACK** (informa bloques recibidos). Ventajas: eficiencia en full-dúplex y **piggyback** para reducir overhead.
- **Wireshark:** ventana calculada = $255 \times 256 = 65\ 280$; flags PSH, ACK; encabezado TCP de 20 bytes.
- **Detección = redundancia.** Errores: de un bit y en ráfaga. Paridad simple (par/impar) y bidimensional; checksum (complemento a 1) y CRC.
- **ARQ:** Stop-and-Wait (1 trama), Go-Back-N (descarta la fallada y todas las posteriores), Selective Repeat (búfer + retransmite solo la fallada).
- **FEC:** corrige en el receptor con redundancia; ventaja = libera tráfico; desventaja = más procesamiento y ancho de banda; **no en LAN**, sí en inalámbricas y gran distancia. Códigos: **Reed-Solomon** (FC, DOCSIS, ráfagas), **LDPC** (alta velocidad, TV satelital), **Polar** (5G), **Turbo** (4G, espacio, señales débiles).